

VOX TECH & WEB ARCHITECTURE — SISTEMA DE INGENIERÍA

MANUAL DE ARQUITECTURA DE DESARROLLO WEB Y ESTÁNDAR DE SISTEMAS

Guía Maestra de Capas, Dev/Ops, Calidad de Código (QA), Seguridad Ampliada, Monitoreo y Reglas de Homologación

Ecosistema: VOX Business Developer Multi-Service Web	Estrategia Git: GitFlow (main, staging, feature/*)	Versión del Estándar: v2.0 — Producción 2026
Despliegue & CI/CD: Vercel Serverless / Automated QA Gate	Seguridad & Datos: Supabase PostgreSQL + RLS + API Shield	Observabilidad: Sentry / Log Centralizado & Telemetría

PRINCIPIO ARQUITECTÓNICO DE DESARROLLO:

Toda página o aplicación desarrollada por VOX debe construirse bajo una arquitectura modular, escalable y desacoplada. Ninguna página web es una isla de código independiente: todas comparten un Núcleo Central de Utilidades (Core Engine), módulos funcionales autónomos y reutilizables, una capa blindada de seguridad y captura de datos (lead-service.js con Cloudflare Turnstile / reCAPTCHA), y un pipeline automatizado de CI/CD que garantiza cero degradación técnica a lo largo del tiempo.

1. Visión del Sistema Web Integrado VOX

El estándar de desarrollo web de VOX establece que cada sitio o landing page de servicio (desarrollo de software, consultoría jurídica, asesoría contable, arquitectura comercial, marketing, etc.) debe funcionar como una extensión natural de un único ecosistema tecnológico.

Se elimina de forma tajante el código 'espagueti', las librerías duplicadas y los formularios desconectados. En su lugar, se implementa una arquitectura por capas estrictamente delimitadas que asegura velocidad de carga ultrarrápida, consistencia visual bajo un Design System unificado, seguridad de datos blindada contra spam/ataques, observabilidad en tiempo real y un mantenimiento ágil a largo plazo.

2. Las Cinco Capas de la Arquitectura Web VOX

- **Capa 1: Núcleo y Fundamentos Compartidos (Core Layer):** Contiene los estilos globales (tokens CSS, variables de diseño, tipografías y reset), librerías base, constantes globales, matriz unificada de variables de entorno (.env) y configuración centralizada de conexión (Supabase client, utilidades de red seguras).
- **Capa 2: Módulos de Componentes Reutilizables (Component Layer):** Bloques de UI estandarizados, accesibles y autónomos: Header/Navbar inteligente con estado activo, Hero sections con call-to-actions dinámicos, Cards de servicios con micro-interacciones, Formularios interactivos con validación instantánea y protección anti-bots (Turnstile/reCAPTCHA v3), Modales modulares, Acordeones FAQ y Footers corporativos.

- **Capa 3: Lógica de Negocio, Servicios y Observabilidad (Service & Business Logic Layer):** Módulos JavaScript independientes y desacoplados del DOM: captura, validación y sanitización universal de leads (lead-service.js), gestión de licencias y créditos (vox-license.js), telemetría, analítica de eventos y monitoreo centralizado de errores (Sentry / LogSnag / Error Boundary).
- **Capa 4: Seguridad, Autenticación y Persistencia Backend (Security & Data Layer):** Integración con backend serverless seguro (/api/*.js) protegido con CORS restrictivo, Rate Limiting por IP (Upstash Redis / Token Bucket), mitigación anti-DDoS, políticas RLS (Row Level Security) en PostgreSQL/Supabase, control de roles (RBAC) y sanitización estricta.
- **Capa 5: Experiencia de Usuario, Cumplimiento y Rendimiento (UX & Performance Layer):** Micro-interacciones optimizadas a 60 FPS, lazy loading nativo de recursos multimedia, versionado de assets para cache-busting inteligente, accesibilidad semántica WCAG/HTML5 (SEO técnico con único <h1> estructurado), Core Web Vitals (< 1.5s LCP), y gestión de privacidad/cookies (Banner de consentimiento GDPR/LGPD compliant).

3. Estructura Estándar de Archivos del Proyecto

Toda implementación web bajo el estándar VOX debe respetar la siguiente organización jerárquica de archivos:

Directorio / Archivo	Capa / Naturaleza	Propósito y Regla de Uso
/assets/css/ (styles.css, vars.css)	Capa 1: Core	Tokens globales de color, tipografía, espaciado y utilidades base. Prohibido estilos inline o CSS aislado no tokenizado.
.env.example / .env.local	Capa 1: Core / Envs	Plantilla y variables locales seguras. Nunca se commitean secretos ni API Keys privadas al repositorio.
/assets/js/supabase-config.js	Capa 1: Core	Conexión única y centralizada con base de datos y autenticación mediante cliente oficial.
/assets/js/lead-service.js	Capa 3: Lógica & Logs	Módulo universal de recepción, validación, telemetría y envío de leads con log centralizado (Sentry/LogSnag).
/assets/js/vox-license.js	Capa 3: Lógica	Control de licenciamiento corporativo, validación de créditos y mecanismos de contingencia/killswitch.
/servicios/*.html	Capa 2: Páginas	Páginas individuales de cada servicio (desarrollo, contable, jurídico, etc.) consumiendo el Core compartido.
/admin/ (dashboard.html, .js)	Capa 4: Gestión	Panel de administración protegido por RLS, tokens JWT y control de acceso basado en roles (RBAC).
/api/ (leads.js, verify-license.js)	Capa 4: Backend	Funciones serverless seguras protegidas con CORS, Rate Limiting por IP y validación de tokens anti-bot.

4. Flujo Dev/Ops, Gestión de Entornos y CI/CD

4.1 Estrategia de Control de Versiones (Git Workflow Estricto):

El repositorio sigue una arquitectura de branching protegida:

- main: Rama exclusiva de Producción. Despliegue automático a producción tras pasar el pipeline de validación.

Protegida contra push directo.

- staging: Rama de Pruebas y Homologación (QA). Utilizada para validación de integración antes de release final.
- feature/* o fix/*: Ramas efímeras de trabajo individual (ej. feature/turnstile-captcha, fix/cors-lead-api). Requieren Pull Request y revisión para integrarse.

4.2 Gestión Segura de Variables de Entorno (Envs):

Para garantizar cero fugas de credenciales en repositorios públicos/privados:

- .env.example: Archivo versionado en Git con las llaves requeridas sin valores sensibles (ej. SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY=xxx).
- .env.local: Archivo local de desarrollo excluido estrictamente en .gitignore.
- .env.production / .env.staging: Variables inyectadas en tiempo de ejecución en la plataforma de hosting (Vercel Project Settings) con segregación estricta entre Service Role Keys (solo backend /api/) y Anon Public Keys (frontend).

4.3 Pipeline de CI/CD e Integración Continua:

Automatización de pruebas y despliegues con GitHub Actions / Vercel Deploy Hooks:

1. Trigger de PR: Ejecución automática de validación de sintaxis (Node syntax check), linter de código y verificación de dependencias.
2. Preview Deployment: Creación automática de entorno de pruebas efímero para validar UI/UX y funcionalidad.
3. Production Gate: El paso a 'main' requiere validación satisfactoria de las pruebas automatizadas y aprobación del Checklist de Homologación.

5. Protocolo de Pruebas, Calidad de Código (QA) y Manejo de Assets

5.1 Linter y Formateador Estándar:

Para mantener consistencia absoluta entre todos los desarrolladores del equipo:

- ESLint & Prettier: Reglas de formateo unificadas (tabulaciones de 2 espacios, comillas simples, punto y coma obligatorio, sin variables huérfanas).
- Stylelint: Validación de tokens CSS evitando el uso de valores hexadecimales 'hardcodeados' fuera de :root.
- Pre-commit Hooks (Husky / Lint-Staged): Verificación obligatoria antes de permitir cualquier 'git commit'.

5.2 Manejo de Caché y Versionado de Assets (Cache-Busting):

- Control de versión en recursos estáticos mediante query params de versión (ej. styles.css?v=2.4) o nombrado con hash (ej. bundle.[hash].js).
- Configuración de encabezados HTTP 'Cache-Control': Recursos inmutables (fuentes, logos e imágenes fijas) con cache prolongado (max-age=31536000, immutable); scripts y HTML con validación 'no-cache' o 'must-revalidate' para reflejar actualizaciones inmediatas sin requerir borrado manual de caché por el cliente.

6. Seguridad Ampliada (Capa 4), Privacidad y Observabilidad (Capa 3)

6.1 Políticas CORS Estrictas y Rate Limiting Serverless:

Todos los endpoints en /api/ (ej. leads.js) deben implementar cabeceras CORS que restrinjan el origen exclusivamente a dominios oficiales de VOX (voxbusinessdeveloper.com y previsualizaciones autorizadas). Asimismo, se implementa Rate Limiting por IP (máximo 5 peticiones de cotización por minuto por IP) para mitigar ataques DDoS, scrapping y spam masivo.

6.2 Protección Anti-Bots y Captcha Inteligente:

Integración obligatoria en todos los formularios públicos (Capa 2/3) de Cloudflare Turnstile o reCAPTCHA v3 invisible. El token generado se valida en el backend serverless (/api/leads.js) previo al almacenamiento en Supabase o despacho de notificaciones vía Resend/Nodemailer.

6.3 Política de Cookies y Cumplimiento Normativo (GDPR / Privacidad):

Implementación estándar de banner de consentimiento de cookies con categorización (Técnicas, Analíticas, Marketing), enlace permanente a la Política de Privacidad y Términos de Servicio, y bloqueo de scripts de tracking de terceros hasta que el usuario otorgue consentimiento explícito.

6.4 Monitoreo Centralizado de Log de Errores y Telemetría:

Integración en lead-service.js y handlers globales de herramientas como Sentry o LogSnag. Cualquier fallo silencioso de red, error 500 de API o excepción no controlada en el cliente se reporta de forma asíncrona con stack trace y contexto (dispositivo, URL de servicio, timestamp) sin degradar la experiencia de usuario.

7. Reglas Técnicas Obligatorias para Nuevas Páginas

- 1. **Cero Código Duplicado (DRY Principle):** Ninguna página debe reescribir lógica de envío de formularios, conexión con Supabase o configuración de APIs; siempre se importan modularmente desde /assets/js/.
- 2. **Estilos Basados en Tokens Globales:** Todos los colores, sombras, tipografías y radios de borde deben provenir de las variables maestras de CSS (:root) para garantizar coherencia y propagación inmediata.
- 3. **Manejo Estandarizado de Formularios y Leads:** Cada formulario debe emitir eventos unificados que capturen: servicio de origen, datos de contacto, requerimiento comercial, token anti-bot y metadatos de trazabilidad.
- 4. **Arquitectura Accesible y Semántica (HTML5):** Uso obligatorio de etiquetas semánticas (<header>, <main>, <section>, <article>, <aside>, <footer>) con un único <h1> por página, estructura jerárquica de encabezados (H2/H3) y atributos ARIA completos.
- 5. **Optimización, Resiliencia y Feedback Visual:** Manejo proactivo de errores (Try/Catch), feedback visual inmediato al usuario en cada acción (estados de carga con spinners, alertas toast de éxito/error) y prevención de render blocking.

8. Checklist de Homologación Pre-Publicación (Gate de Producción)

Antes de que cualquier página, servicio o módulo desarrollado por VOX sea publicado en el entorno de producción, debe superar y documentar el siguiente checklist técnico:

Estado	Área de Verificación	Criterio de Aceptación Obligatorio
[OK]	Core & Arquitectura	Importación correcta del Core de estilos compartidos, tokens globales y scripts sin dependencias huérfanas.
[OK]	Lead Service & Captcha	Formulario de contacto conectado con lead-service.js, protección anti-bot (Turnstile/reCAPTCHA) y registro en Supabase/Email.
[OK]	DevOps & Variables (Envs)	Variables de entorno segregadas (.env.example actualizado, claves privadas configuradas en Vercel y ausentes en el frontend).
[OK]	Calidad & Linters (QA)	Código validado sin errores de sintaxis AST, formateado bajo estándar ESLint/Prettier y con estrategia de cache-busting en assets.

[OK]	Seguridad Backend & CORS	Endpoints /api/ con CORS restringido a dominios oficiales y Rate Limiting por IP configurado.
[OK]	Monitoreo de Errores	Integración de logging de excepciones (Sentry/LogSnag) y manejo resiliente de fallos de red.
[OK]	SEO & Accesibilidad Semántica	Metaetiquetas OpenGraph completas, Title descriptivo, Meta Description optimizada, único <h1> por página y jerarquía H2/H3.
[OK]	Rendimiento (Performance)	Tiempos de carga menores a 1.5 - 2.0 segundos en conexiones móviles/estándar (Core Web Vitals verdes).
[OK]	Responsive Design	Adaptabilidad móvil verificada en breakpoints estándar (360px, 768px, 1024px, 1440px) sin desbordamientos horizontales.
[OK]	Consola Limpia & Privacidad	Consola del navegador sin advertencias o errores críticos y banner de consentimiento de cookies funcional.

9. Dictamen y Evaluación de la Arquitectura VOX

EVALUACIÓN DE INGENIERÍA & IMPACTO EN EL NEGOCIO:

1. Arquitectura Altamente Escalable:

La división modular en 5 capas claras (Core, Componentes, Lógica de Servicios, Seguridad/Datos y UX/Rendimiento) garantiza un mantenimiento eficiente a largo plazo, desacopla el ciclo de vida del frontend y backend, y evita que la base de código se degrade con el tiempo ante el crecimiento del catálogo de servicios.

2. Enfoque Orientado a Producto y Conversión Comercial:

El hecho de estandarizar la captura de leads mediante un servicio universal (lead-service.js) desacoplado de las páginas individuales asegura que no se pierda información comercial en ningún punto del ecosistema, unificando trazabilidad, analítica y seguridad en un solo flujo.